

Skill-14

```
GNU nano 8.7.1 skill_14.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

void execute_command(char *input)
{
    char *args[MAX_ARGS];
    int count = 0;
    int fd = -1;

    char *redirect = strchr(input, '>');

    if (redirect != NULL)
    {
        *redirect = '\\0';
        redirect++;

        while (*redirect == ' ')
            redirect++;

        char *filename = strtok(redirect, " ");

        if (filename == NULL)
        {
            printf("Missing output file\\n");
            return;
        }

        fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd == -1)
        {
            perror("open");
            return;
        }
    }

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        if (fd != -1)
        {
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    waitpid(pid, NULL, 0);

    if (fd != -1)
        close(fd);
}
```

```

int main()
{
    char input[MAX_INPUT];

    printf("==== Skill 14 =====\n");
    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    execute_command(input);

    return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute ^{^C} Location
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify ^{^/} Go To Lin

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_14.c -o skill_14
```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_14
```

```
==== Skill 14 =====
```

```
Enter command: ls > output.txt
```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ cat output.txt
```

```

a.out
output.txt
program1
program1.c
program2
program2.c
shellforge
shellforge.c
skill_10
skill_10.c
skill_11
skill_11.c
skill_12
skill_12.c
skill_13
skill_13.c
skill_14
skill_14.c
skill_3
skill_3.c
skill_4
skill_4.c
skill_5
skill_5.c
skill_6
skill_6.c
skill_7
skill_7.c
skill_8
skill_8.c
skill_9
skill_9.c
temp

```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ |
```

Skill-15

```
sahasrasagi@Sahasra: ~/OS_L × + v
GNU nano 8.7.1 skill_1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 500
#define MAX_COMMANDS 10
#define MAX_ARGS 30

void parse_args(char *command, char **args)
{
    int i = 0;
    char *token = strtok(command, " ");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;
}

void execute_pipeline(char *input)
{
    char *commands[MAX_COMMANDS];
    int command_count = 0;

    char *token = strtok(input, "|");

    while (token != NULL && command_count < MAX_COMMANDS)
    {
        commands[command_count++] = token;
        token = strtok(NULL, "|");
    }

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }
}
```

```

    for (int i = 0; i < command_count - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < command_count; i++)
    {
        waitpid(pids[i], NULL, 0);
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("==== Skill 15 =====\n");
    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    execute_pipeline(input);

    return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ nano skill_15.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_15.c -o skill_15
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_15
==== Skill 15 =====
Enter command: ls | grep .c > output.txt
sahasrasagi@Sahasra:~/OS_Lab/skill$ cat output.txt
program1.c
program2.c
shellforge.c
skill_10.c
skill_11.c
skill_12.c
skill_13.c
skill_14.c
skill_15.c
skill_3.c
skill_4.c
skill_5.c
skill_6.c
skill_7.c
skill_8.c
skill_9.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ |

```

Skil-16:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 20
#define MAX_COMMAND 200

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    jobs[job_count].running = 1;

    printf("[%d] %d running %s\n",
           jobs[job_count].job_id,
           pid,
           command);

    job_count++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;
            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

            if (result > 0)
            {
                jobs[i].running = 0;
            }
        }
    }
}

void display_jobs()
{
    update_jobs();

    printf("\nActive Jobs:\n");

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] Running PID=%d %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
    }
}
```

```

int main()
{
    char input[MAX_COMMAND];

    printf("==== Skill 16 ==== \n");

    while (1)
    {
        update_jobs();

        printf("\nmyshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "jobs") == 0)
        {
            display_jobs();
        }
        else if (strcmp(input, "clearjobs") == 0)
        {
            remove_completed_jobs();
            printf("Completed jobs removed\n");
        }
        else if (strcmp(input, "exit") == 0)
        {
            break;
        }
        else
        {
            int length = strlen(input);

            if (length > 0 && input[length - 1] == '&')
            {
                input[length - 1] = '\0';

                while (length > 1 && input[length - 2] == ' ')
                {
                    input[length - 2] = '\0';
                    length--;
                }

                start_background_job(input);
            }
            else
            {
                printf("Use '&' to run a background job\n");
            }
        }
    }

    return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ nano skill_16.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_16.c -o skill_16
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_16
==== Skill 16 ====

myshell> sleep 20
Use '&' to run a background job

myshell> sleep 20 &
[1] 1006 running  sleep 20

myshell> |

```